

JVM Tuning and Spring Boot Microservice Performance

John Michael Bilbao

[Course / Section]

August 10, 2026

Introduction

A microservice is just a small application that does one job and talks over HTTP. I used Spring Boot for this one because it comes with Tomcat built in and handles most of the setup itself, so the whole service is only a few lines of code.

Spring Boot runs on the JVM, and the JVM is what decides how much memory the application gets and when it clears out objects that are no longer needed. That clearing out is garbage collection. If the heap is too small the collector keeps running, and the application spends more time freeing memory than answering requests. Tuning means setting options like the heap size and the collector type to fit the work the application actually does. I built a small item service, put it under load, watched it in VisualVM, then changed the settings and measured what happened. Everything here ran on Java 21 with Spring Boot 3.4.1, on a machine with 8 cores and 31 GB of RAM.

The microservice

It is a Maven project with the Spring Web and Actuator dependencies. Three classes, and the controller is the only interesting one:

```
@RestController
@RequestMapping("/api/items")
public class ItemController {

    private static final String[] CATEGORIES =
        {"Tools", "Books", "Food", "Toys", "Parts"};

    @GetMapping
    public List<Item> getItems(@RequestParam(defaultValue = "5000") int count) {
        List<Item> items = new ArrayList<>();
        for (int i = 0; i < count; i++) {
            String name = "Item " + i + " - " + CATEGORIES[i % CATEGORIES.length];
            items.add(new Item(i, name,
                CATEGORIES[i % CATEGORIES.length], 10.0 + (i % 100)));
        }
        return items;
    }
}
```

Item is a record with an id, name, category and price. The endpoint is GET /api/items?count=5000 and it returns the list as JSON.

The list gets rebuilt on every request instead of being cached, which is on purpose. A cached list would barely allocate anything and there would be nothing to watch in VisualVM. Rebuilding it means every request throws away thousands of objects.



```
itemservice endpoint test
$ curl -s "http://localhost:8085/api/items?count=2" | python3 -m json.tool
[
  {
    "id": 0,
    "name": "Item 0 - Tools",
    "category": "Tools",
    "price": 10.0
  },
  {
    "id": 1,
    "name": "Item 1 - Books",
    "category": "Books",
    "price": 11.0
  }
]

$ curl -s -o /dev/null -w "HTTP %{http_code}  %{size_download} bytes  %{time_total}s\n" "http://localhost:8085/api/items?count=5000"
HTTP 200 349281 bytes 0.043315s

$ curl -s http://localhost:8085/actuator/health
{"status":"UP"}
```

Figure 1: The endpoint responding, and the health check

Monitoring with VisualVM

I got VisualVM from visualvm.github.io and started it while the service was already running. The process shows up on the left under Local, and double clicking it opens the Overview tab, which is handy because it shows the JVM options the process actually started with. I ran the first round with a small heap and the serial collector so there would be something to find:

```
java -Xms64m -Xmx128m -XX:+UseSerialGC \
-cp "target/classes:$(cat cp.txt)" \
com.example.itemservice.ItemserviceApplication
```

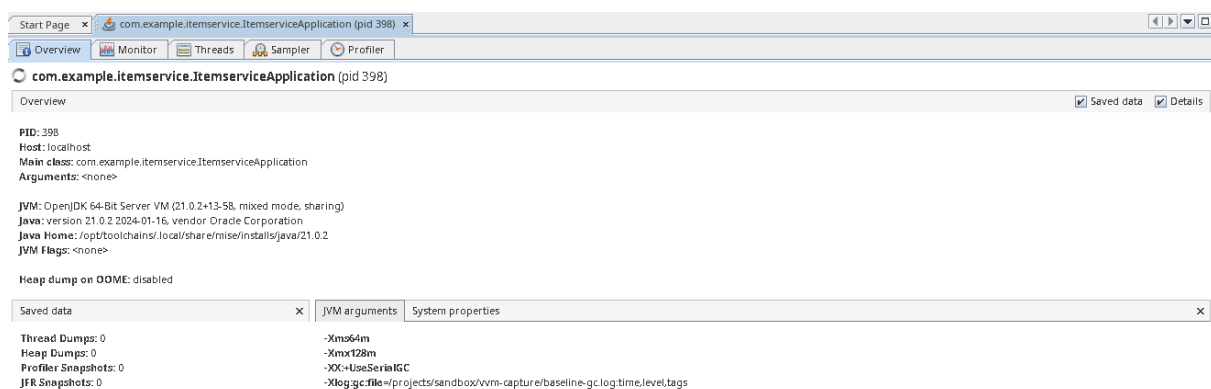


Figure 2: Overview tab, showing the baseline flags in effect

Nothing much happens while the application is idle, so I wrote a Python script that hits the endpoint with 16 threads asking for 25,000 items each, and left it looping for about two and a half minutes while I watched the Monitor tab.

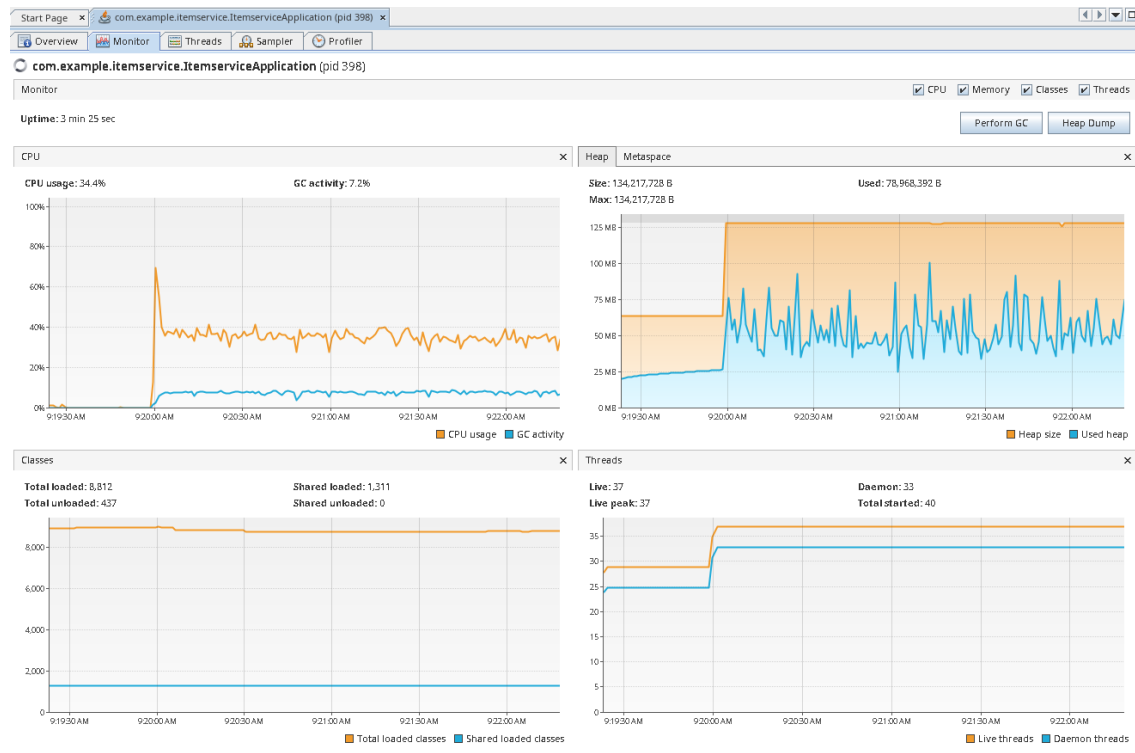


Figure 3: Monitor tab during the baseline run

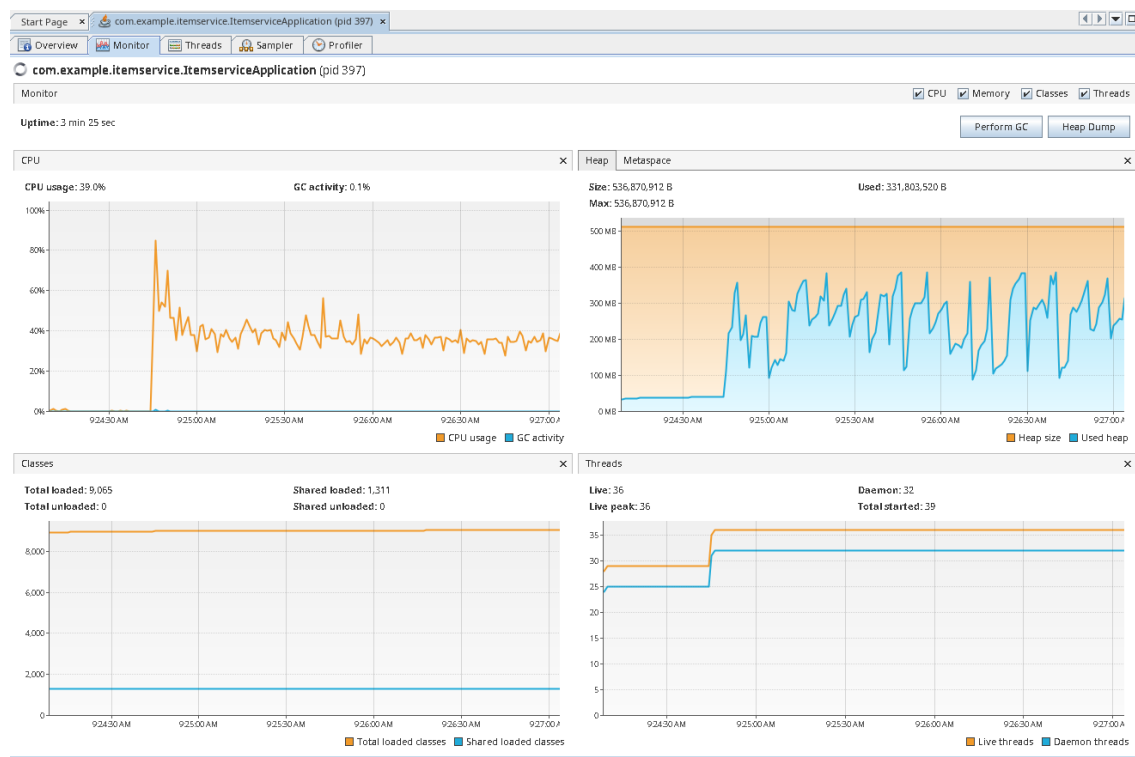
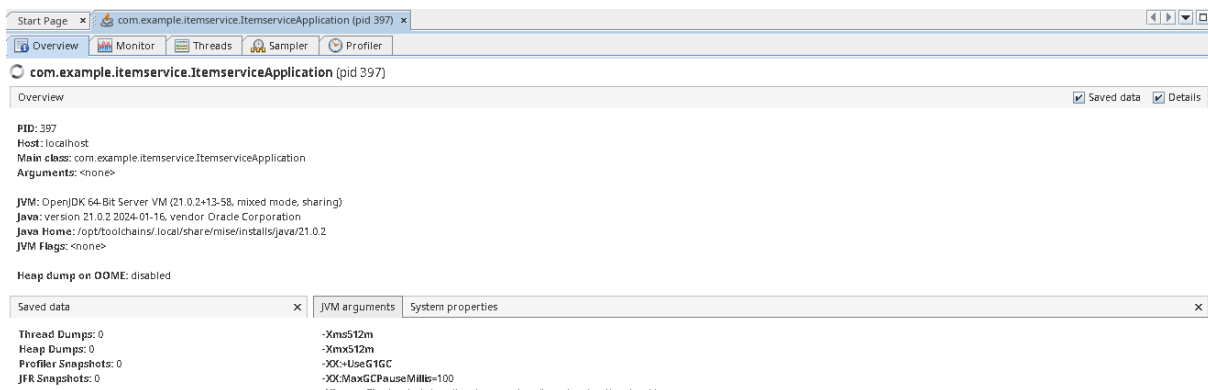
Heap went straight to the 128 MB ceiling and stayed pinned there. Used heap sawtoothed between roughly 25 and 100 MB and never settled, so the collector was running constantly rather than every now and then. CPU sat around 34 percent, and threads went from 29 to 37 when the load arrived and then stayed flat, so the thread count was not the problem.

The GC activity figure confused me at first. VisualVM showed 7.2 percent, which sounds harmless, but the GC log for the same session recorded 3,137 collections, 695 of them full GCs, and 95.5 seconds of pauses. During the busiest ten seconds the application was frozen 68 percent of the time. Both numbers are right. Serial GC stops everything and then works on a single thread, and VisualVM reports GC time spread across all 8 cores, so one fully busy core comes out at around 7 percent. The graph answers how much of the CPU is going into GC, and the log answers how long the application is frozen. For a service that has to answer requests, the second one is what matters.

Changing the settings

The heap was the problem, not CPU and not threads. I raised the maximum from 128 MB to 512 MB and set the minimum to match, which stops the JVM growing the heap while it is already busy. I also swapped the serial collector for G1, since G1 spreads its work across cores and does much of it concurrently, and gave it a 100 ms pause target.

```
java -Xms512m -Xmx512m -XX:+UseG1GC -XX:MaxGCPauseMillis=100 \
  -cp "target/classes:$(cat cp.txt)" \
  com.example.itemservice.ItemserviceApplication
```



Used heap now moves between about 100 and 400 MB with visible gaps instead of one solid band, so the collector gets to rest between cycles. GC activity dropped from 7.2 percent to 0.1, full GCs went from 695 to none, and the total pause time fell from 95.5 seconds to 2.3. CPU actually went up a little, from 34 to 39 percent, which looked wrong until I realised the CPU was finally being spent on requests instead of on collecting garbage.

Results

Screenshots show behaviour but not speed, so I ran the load test again with VisualVM detached: 1,200 requests, 16 at a time, 25,000 items each. Same code and same machine, only the flags were different.

	Before	After
Requests per second	103.0	173.5
Average response time	155.0 ms	92.0 ms
95th percentile response time	287.7 ms	115.3 ms
Time to finish 1,200 requests	11.65 s	6.92 s
Collections	369	32
Full GCs	76	0
Total pause time	9,812 ms	191 ms

76 full GCs before and none after is the number that stands out. Each one freezes every thread, and that is what made the slowest requests slow, which is why the 95th percentile came down from 287.7 ms to 115.3 ms.

Two things I had to be careful about when reading the log. The pause total covers the 18.9 seconds the log spans rather than just the 11.65 seconds of measured requests, so it works out to 52 percent of that window and not the much higher figure I got the first time. And G1 logs its concurrent phases alongside its pauses, and concurrent work does not stop the application, so counting those would have made the tuned total look about 45 ms worse than it really is.

Earlier I had run a lighter test, 2,000 requests of 5,000 items with 8 threads, and it barely moved: 577 requests per second before against 571 after, and 13.8 ms average response time against 14.0 ms. Collections still dropped from 160 to 14, but the application was not waiting on garbage collection at that load in the first place, so tidying up GC bought nothing. The 68 percent gain only turned up once the load was heavy enough to actually run the small heap out of room.

What I learned

Tuning only helps if the thing you are tuning is what is actually slowing you down. The light load test made that obvious, since GC work dropped a long way and the response times did not move at all.

It is worth checking what a percentage is a percentage of. The 7.2 percent GC activity looked fine sitting next to a log that showed the application frozen 68 percent of the time, and neither number was wrong.

Repeated full GCs are a better warning sign than a heap sitting near its limit. A full heap can just mean the collector is doing its job well. Full GCs freeze everything, every time.

Bigger is not automatically better either. I went with 512 MB because that was what got rid of the full GCs, and going much higher would waste memory and can make individual collections take longer. Java's own default on this machine was a 7.9 GB heap with G1 already chosen, and the endpoint never struggled, so I had to shrink the heap on purpose to create a problem worth looking at. That is roughly the situation you would be in deploying into a container with a small memory limit.